Final Terraform Task.

1. Create VPC.
2. Create Internet gateway.
3. Create Custom Route Table.
4. Create Subnet.
5. Associate subnet with Route Table.
6. Create Security Group to allow port 22.80,443.
7. Create a network interface with an ip in the subnet that was created in step 4.
8. Assign an elastic IP to the network interface created in step 7.
9. Create Ubuntu server and install/enable apache2.

Terraform Final Task – AWS Infrastructure Deployment

Project Objective

The objective of this assignment is to create AWS infrastructure using Terraform and deploy an Ubuntu EC2 web server with Apache2.

We use Terraform to automate the creation of networking resources, security rules, a network interface, an Elastic IP and an EC2 instance.

After deployment, we verify that Apache2 is running and accessible through the public IP address.

Project Environment

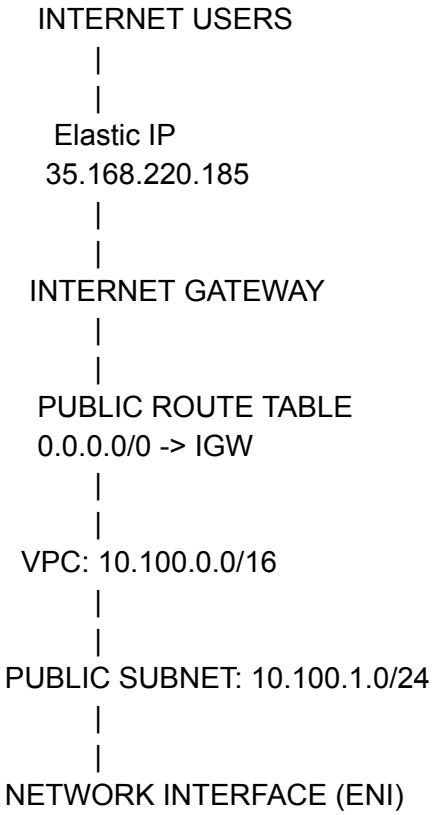
Item	Configuration
Cloud provider	AWS
AWS Region	us-east-1
AWS CLI profile	terraform-lab
Terraform version	1.17.0

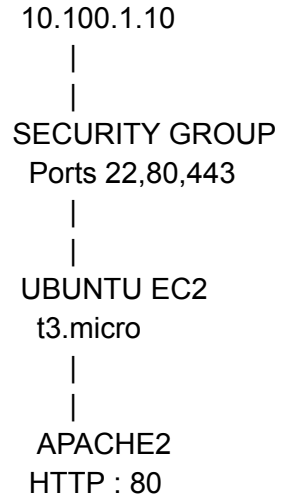
AWS provider	hashicorp/aws v6.71.0
Operating system	Ubuntu 24.04 LTS
Instance type	t3.micro
EC2 key pair	my_server
VPC CIDR	10.100.0.0/16
Subnet CIDR	10.100.1.0/24
Private IP	10.100.1.10
Elastic IP	35.168.220.185

4. Architecture Explanation

The architecture contains a VPC, public subnet, Internet Gateway, route table, security group, network interface, Elastic IP and Ubuntu EC2 instance.

Traffic flow





How it works:

1. A user sends a request to the public IP address.
2. The Elastic IP is associated with the EC2 instance's network interface.
3. The Internet Gateway provides internet connectivity to the VPC.
4. The route table directs internet-bound traffic to the Internet Gateway.
5. The security group permits the configured traffic.
6. The network interface connects the EC2 instance to the subnet.
7. Apache2 receives the HTTP request and returns the web page.

---

5. Project Folder Structure

Create one folder for the Terraform project.

Windows PowerShell:

New-Item -ItemType Directory -Force -Path "C:\Users\HP\terraform-final-task"

Set-Location "C:\Users\HP\terraform-final-task"

Create these Terraform files:

terraform-final-task/
│
├── versions.tf
├── provider.tf
├── main.tf
├── security.tf

```
├── network_interface.tf
├── eip.tf
├── ec2.tf
├── outputs.tf
└── .gitignore
```

Purpose of each file

File	Purpose
versions.tf	Defines Terraform and AWS provider versions
provider.tf	Configures AWS region and profile
main.tf	Creates VPC and networking resources
security.tf	Creates security group and firewall rules
network_interface.tf	Creates the network interface
eip.tf	Creates and associates Elastic IP
ec2.tf	Creates Ubuntu EC2 and installs Apache2
outputs.tf	Displays resource IDs and public URL
.gitignore	Excludes state and sensitive files from Git

---

6. Complete Terraform Code

File 1: versions.tf

Purpose: This file specifies the minimum Terraform version and the AWS provider version required for the project.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
```

```
  }
}
```

Explanation:

- `terraform`: Defines Terraform configuration.
- `required_version`: Requires Terraform 1.5.0 or later.
- `required_providers`: Defines the providers required.
- `source`: Specifies the official AWS provider.
- `version`: Allows compatible AWS provider 6.x releases.

File 2: provider.tf

Purpose: Configures Terraform to connect to AWS.

```
provider "aws" {
  region  = "us-east-1"
  profile = "terraform-lab"
}
```

Explanation:

- `provider "aws"`: Uses AWS as the cloud provider.
- `region`: Creates resources in the North Virginia region.
- `profile`: Uses the AWS CLI profile named `terraform-lab` for authentication.

File 3: main.tf

Purpose: Creates the VPC, Internet Gateway, custom route table, public subnet and route table association.

```
# 1. Create VPC
resource "aws_vpc" "main" {
  cidr_block           = "10.100.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-final-vpc"
  }
}

# 2. Create Internet Gateway
```

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "terraform-final-igw"
  }
}

# 3. Create Custom Route Table
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "terraform-final-public-rt"
  }
}

# 4. Create Public Subnet
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.100.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "terraform-final-public-subnet"
  }
}

# 5. Associate Subnet with Route Table
resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

Explanation of main.tf

Step 1 – VPC

resource "aws_vpc" "main"

Creates a Virtual Private Cloud.

- CIDR `10.100.0.0/16` defines the VPC IP address range.
- DNS support and DNS hostnames are enabled.
- The Name tag identifies the VPC in AWS.

Step 2 – Internet Gateway

resource "aws_internet_gateway" "igw"

Creates an Internet Gateway and attaches it to the VPC.

The Internet Gateway allows communication between the VPC and the internet when routing and security rules permit it.

Step 3 – Custom Route Table

resource "aws_route_table" "public"

Creates a custom route table.

The route:

```
route {
  cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.igw.id
}
```

Means that traffic destined for any IPv4 address outside the VPC is directed to the Internet Gateway.

Step 4 – Public Subnet

resource "aws_subnet" "public"

Creates a subnet inside the VPC.

- Subnet CIDR: `10.100.1.0/24`
- Availability Zone: `us-east-1a`
- `map_public_ip_on_launch = true` enables automatic public IPv4 assignment for instances launched in this subnet, unless overridden.

Step 5 – Route Table Association

resource "aws_route_table_association" "public"

Associates the public subnet with the custom route table.

Without this association, the subnet would not use the custom route table created above.

File 4: security.tf

Purpose: Creates a security group to control incoming and outgoing traffic.

```
# 6. Create Security Group
resource "aws_security_group" "web" {
  name        = "terraform-final-web-sg"
  description = "Allow SSH, HTTP and HTTPS traffic"
  vpc_id      = aws_vpc.main.id

  # SSH access
  ingress {
    description = "SSH access"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["124.123.159.170/32"]
  }

  # HTTP access
  ingress {
    description = "HTTP access"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # HTTPS access
  ingress {
    description = "HTTPS access"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Allow outbound traffic
  egress {
    from_port   = 0
```

```
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "terraform-final-web-sg"
  }
}
```

Explanation

A security group acts as a virtual firewall for the network interface or EC2 instance.

Port	Protocol	Purpose	Source
22	TCP	SSH remote access	Administrator IP only
80	TCP	HTTP website access	0.0.0.0/0
443	TCP	HTTPS traffic	0.0.0.0/0
All outbound	All	Outgoing connections	0.0.0.0/0

Important: SSH is restricted to `124.123.159.170/32`, the public IP used during this lab. If your public IP changes, update this rule before expecting SSH access.

Port 443 is allowed by the security group, but Apache was configured for HTTP only. HTTPS requires a certificate and Apache SSL configuration.

File 5: network_interface.tf

Purpose: Creates a network interface in the public subnet with a fixed private IP address.

```
# 7. Create Network Interface
resource "aws_network_interface" "web" {
  subnet_id       = aws_subnet.public.id
  private_ips     = ["10.100.1.10"]
  security_groups = [aws_security_group.web.id]

  tags = {
    Name = "terraform-final-web-eni"
  }
}
```

Explanation

- `aws_network_interface`: Creates an Elastic Network Interface (ENI).
- `subnet_id`: Places the ENI inside our public subnet.
- `private_ips`: Assigns the fixed private IP `10.100.1.10`.
- `security_groups`: Attaches the security group to the ENI.

An ENI acts like a virtual network card for an EC2 instance.

File 6: eip.tf

Purpose: Allocates an Elastic IP and associates it with the network interface.

```
# 8. Allocate and associate Elastic IP
resource "aws_eip" "web" {
  domain                    = "vpc"
  network_interface         = aws_network_interface.web.id
  associate_with_private_ip = "10.100.1.10"

  tags = {
    Name = "terraform-final-web-eip"
  }
}
```

Explanation

- `aws_eip`: Creates an Elastic IP.
- `domain = "vpc"`: Allocates an Elastic IP for use in a VPC.
- `network_interface`: Associates the Elastic IP with our ENI.
- `associate_with_private_ip`: Specifies which private IP on the ENI receives the association.

Our Elastic IP is:

35.168.220.185

This public IP allows users to reach the web server from the internet.

File 7: ec2.tf

Purpose: Fetches the Ubuntu AMI, creates the EC2 instance and installs Apache2 automatically.

```
# Get the current Ubuntu 24.04 LTS AMI ID
data "aws_ssm_parameter" "ubuntu_ami" {
  name =
"/aws/service/canonical/ubuntu/server/24.04/stable/current/amd64/hvm/ebs-gp3/ami-id"
}

# 9. Create Ubuntu EC2 Instance
resource "aws_instance" "web" {
  ami           = data.aws_ssm_parameter.ubuntu_ami.value
  instance_type = "t3.micro"
  key_name      = "my_server"

  # Attach existing ENI as primary network interface
  primary_network_interface {
    network_interface_id = aws_network_interface.web.id
  }

  # Install and configure Apache2
  user_data = <<-EOF
    #!/bin/bash
    set -eux

    apt-get update
    DEBIAN_FRONTEND=noninteractive apt-get install -y apache2

    systemctl enable apache2
    systemctl start apache2

    echo '<h1>Terraform Final Task - Apache is running!</h1>' > /var/www/html/index.html
  EOF

  user_data_replace_on_change = true

  root_block_device {
    volume_size           = 8
    volume_type           = "gp3"
    encrypted             = true
    delete_on_termination = true
  }

  tags = {
    Name = "terraform-final-ubuntu-web"
  }
}
```

Explanation of ec2.tf

1. Ubuntu AMI data source

data "aws_ssm_parameter" "ubuntu_ami"

Retrieves the current Ubuntu 24.04 LTS AMI ID from AWS Systems Manager Parameter Store.

We use a data source because we want to read an existing AWS value rather than create a new resource.

2. EC2 instance

resource "aws_instance" "web"

Creates the Ubuntu EC2 instance.

- `ami`: Uses the Ubuntu AMI ID retrieved from SSM.
- `instance_type`: Uses a t3.micro instance.
- `key_name`: Uses the existing EC2 key pair named `my_server`.

3. Primary network interface

```
primary_network_interface {
  network_interface_id = aws_network_interface.web.id
}
```

Attaches the ENI created in `network_interface.tf` as the instance's primary network interface.

This allows the instance to use the fixed private IP and security group already configured on that ENI.

4. User data

The user-data script runs during the EC2 instance's first boot.

```
#!/bin/bash
set -eux
```

Starts a Bash script and enables command tracing and exit-on-error behavior.

apt-get update

Updates Ubuntu's package lists.

DEBIAN_FRONTEND=noninteractive apt-get install -y apache2

Installs Apache2 without interactive prompts.

systemctl enable apache2

Configures Apache2 to start automatically when the server boots.

systemctl start apache2

Starts Apache2 immediately.

echo '<h1>Terraform Final Task - Apache is running!</h1>' > /var/www/html/index.html

Creates the homepage that we use to test the web server.

5. Root EBS volume

```
root_block_device {
  volume_size           = 8
  volume_type           = "gp3"
  encrypted             = true
  delete_on_termination = true
}
```

Creates an encrypted 8-GB gp3 root disk and configures it to be deleted when the EC2 instance is terminated.

File 8: outputs.tf

Purpose: Displays useful information after Terraform creates the infrastructure.

```
output "vpc_id" {
  value = aws_vpc.main.id
}

output "internet_gateway_id" {
  value = aws_internet_gateway.igw.id
}

output "route_table_id" {
  value = aws_route_table.public.id
}

output "public_subnet_id" {
  value = aws_subnet.public.id
}
```

```
output "public_subnet_cidr" {
  value = aws_subnet.public.cidr_block
}

output "security_group_id" {
  value = aws_security_group.web.id
}

output "network_interface_id" {
  value = aws_network_interface.web.id
}

output "network_interface_private_ip" {
  value = aws_network_interface.web.private_ip
}

output "elastic_ip" {
  value = aws_eip.web.public_ip
}

output "elastic_ip_allocation_id" {
  value = aws_eip.web.allocation_id
}

output "ec2_instance_id" {
  value = aws_instance.web.id
}

output "apache_url" {
  value = "http://${aws_eip.web.public_ip}"
}
```

Explanation: Outputs allow us to see the resource IDs, IP addresses and application URL after deployment.

File 9: .gitignore

Purpose: Prevents Terraform state and sensitive variable files from being committed to Git.

```
.terraform/
*.tfstate
*.tfstate.*
crash.log
```

crash.*.log
*.tfvars
*.tfvars.json
override.tf
override.tf.json
*_override.tf
*_override.tf.json

---

7. Terraform Deployment Steps

After creating all the files, open PowerShell in the project folder.

Step 1 – Initialize Terraform

terraform init

What happens?

Terraform initializes the project and downloads the AWS provider plugin.

Expected message:

Terraform has been successfully initialized!

Step 2 – Format Terraform files

terraform fmt

Formats the Terraform code files consistently.

Step 3 – Validate the configuration

terraform validate

Checks whether the Terraform configuration is syntactically valid.

Expected:

Success! The configuration is valid.

Step 4 – Review the execution plan

terraform plan

Terraform compares the configuration with the current state and AWS infrastructure.

It displays which resources Terraform plans to create, update or destroy.

Review the plan before continuing.

Step 5 – Create the infrastructure

terraform apply

Terraform displays the proposed actions and asks for confirmation.

Enter:

yes

Terraform then creates the resources.

In this project, the resources were applied in stages:

- First apply: 5 networking resources.
- Second apply: Security Group.
- Third apply: Network Interface.
- Fourth apply: Elastic IP.
- Fifth apply: Ubuntu EC2 instance.

The final EC2 apply completed successfully:

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Step 6 – View Terraform outputs

terraform output

This displays the IDs and IP addresses of the created AWS resources.

---

8. Final Deployment Results

The following resources were created successfully.

Resource	AWS ID / Value
VPC	vpc-0a37fd33813d62fcc
Internet Gateway	igw-0904f046726653137

Route Table	rtb-01e7e338290b704ca
Public Subnet	subnet-04c5ab259d6b0a40a
Security Group	sg-0dc115ea8390e85a2
Network Interface	eni-0d43646d165f8b76e
ENI Private IP	10.100.1.10
Elastic IP	35.168.220.185
EC2 Instance	i-0f81be764f2958b9a

9. Apache2 Verification

After the EC2 instance was created, we tested the public IP using Windows PowerShell.

Command:

Invoke-WebRequest -Uri "http://35.168.220.185" -UseBasicParsing

Successful response:

StatusCode : 200
StatusDescription : OK

Content:
<h1>Terraform Final Task - Apache is running!</h1>

Result: HTTP 200 OK confirms that the web server is reachable and Apache2 returned the expected homepage.

Open this URL in a browser:

http://35.168.220.185

10. Troubleshooting Notes

Issue 1 – Sensitive AMI output

Terraform initially reported an error when exporting the Ubuntu AMI ID because the value was marked sensitive.

Resolution: We removed the unnecessary `ubuntu_ami_id` output. The SSM data source remains in `ec2.tf` and continues to provide the AMI ID to the EC2 resource.

Issue 2 – Deprecated network interface argument

Terraform warned that the older `network_interface` block was deprecated for the primary interface.

Resolution: We used the `primary_network_interface` block instead.

Issue 3 – Apache is not immediately accessible

EC2 creation can complete before the user-data script finishes.

Resolution:

- Wait a few minutes.
- Check that the instance is running.
- If SSH is available, inspect `/var/log/cloud-init-output.log`.
- Check Apache with `sudo systemctl status apache2`.

---

11. Cleanup – Destroy AWS Resources

After completing the assignment and taking screenshots, remove the resources if you no longer need them.

First review:

terraform plan -destroy

Then run:

terraform destroy

Review the destroy plan carefully and enter `yes` only if it lists the resources belonging to this final task.

Terraform will remove the resources managed by this project's state, including the EC2 instance, Elastic IP, ENI, security group and networking resources.

Important: Do not delete unrelated AWS resources, especially the separate Terraform-05 backend S3 bucket and DynamoDB table.

---

12. Interview Explanation

If an interviewer asks, “What did you do in your Terraform final task?”, you can explain:

“I used Terraform to provision AWS infrastructure from scratch. First, I created a custom VPC, Internet Gateway, route table and public subnet. Then I associated the subnet with the route table and created a security group to allow SSH, HTTP and HTTPS traffic.

After that, I created a network interface with a fixed private IP address and associated an Elastic IP with it. Finally, I launched an Ubuntu 24.04 EC2 instance using that network interface.

I used EC2 user data to automatically update Ubuntu, install Apache2, enable the service and start it. After deployment, I verified the web server using its public IP address and received an HTTP 200 OK response.”

---

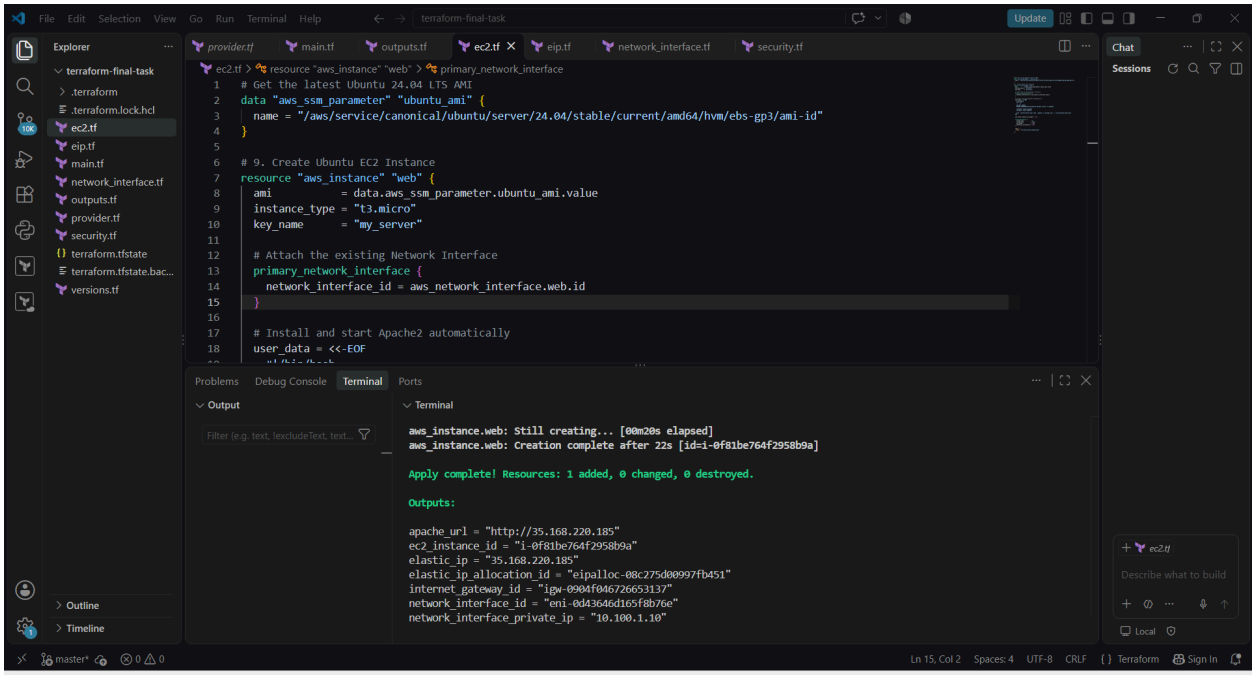
13. Final Conclusion

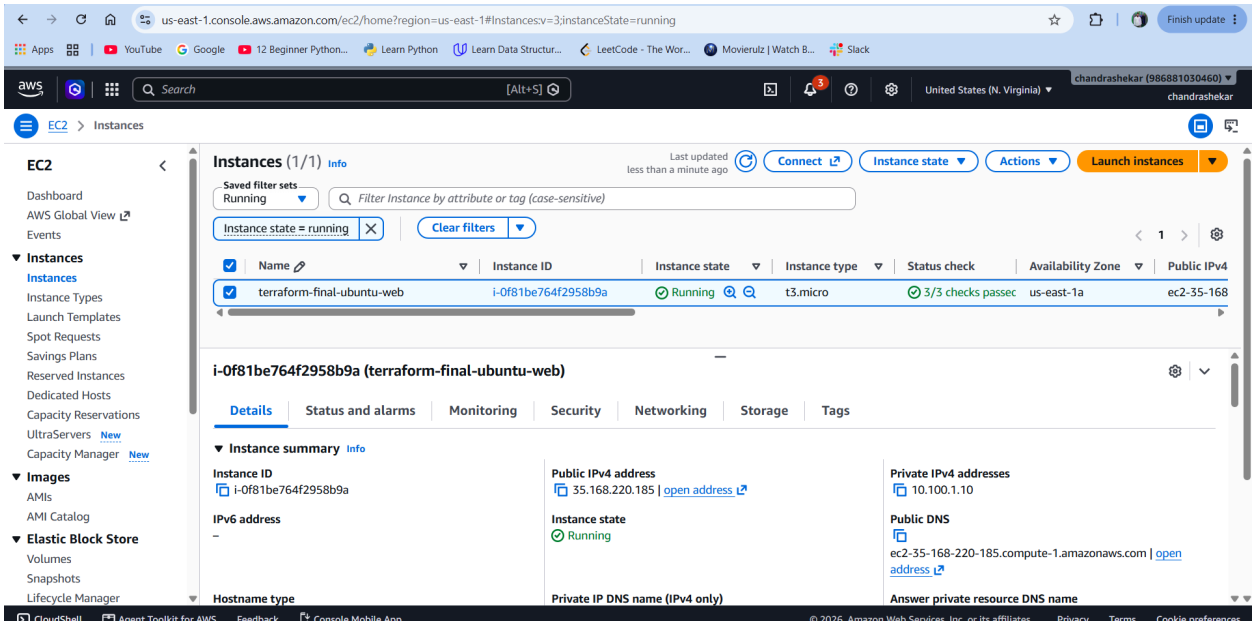
In this assignment, we successfully automated AWS infrastructure provisioning using Terraform.

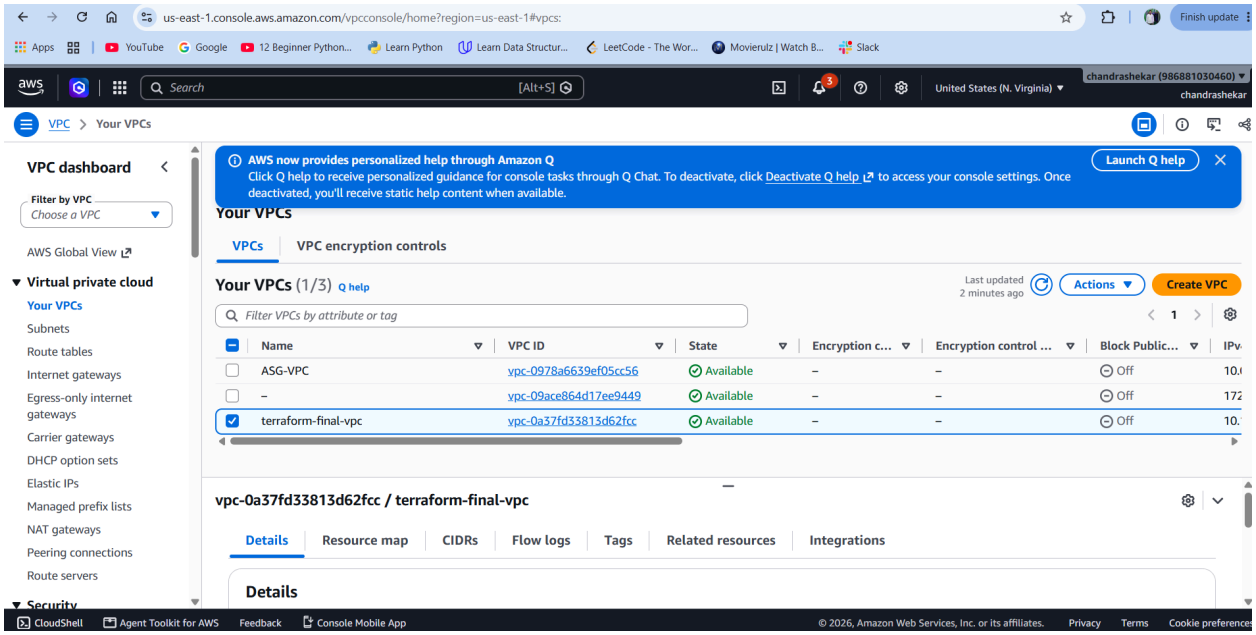
We learned how to:

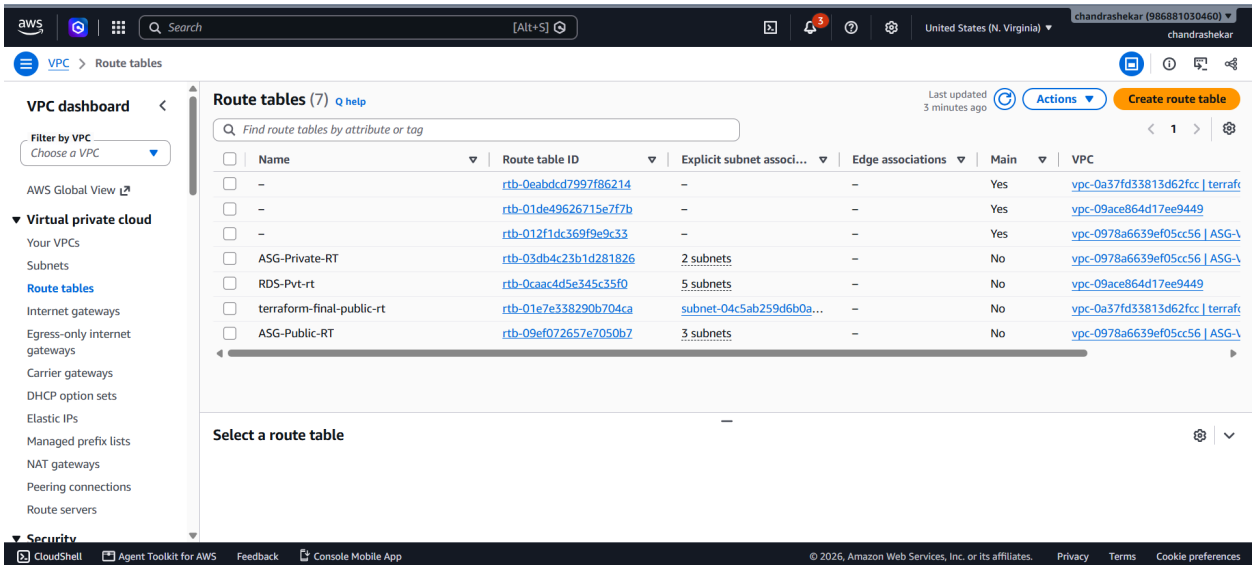
1. Create a custom VPC and public networking.
2. Configure an Internet Gateway and route table.
3. Create and associate a public subnet.
4. Configure security group rules.
5. Create a network interface with a fixed private IP.
6. Allocate and associate an Elastic IP.
7. Launch an Ubuntu EC2 instance.
8. Install and configure Apache2 using user data.
9. Verify the deployed web application using its public IP.

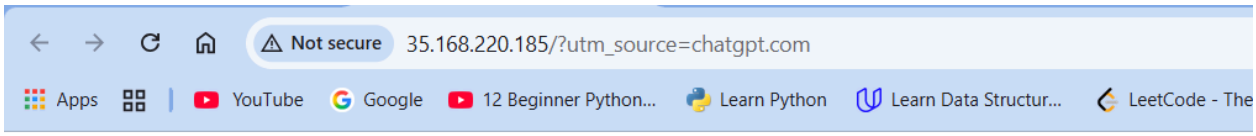
Final result: All nine assignment steps were completed successfully, and Apache2 was verified with HTTP 200 OK.


terraform-final-task
Explorer
.terraform
.terraform.lock.hcl
ec2.tf
eip.tf
main.tf
network_interface.tf
outputs.tf
provider.tf
security.tf
terraform.tfstate
terraform.tfstate.bac...
versions.tf
Outline
Timeline
provider.tf
main.tf
outputs.tf
ec2.tf
eip.tf
network_interface.tf
security.tf
ec2.tf > resource "aws_instance" "web" > primary_network_interface
Get the latest Ubuntu 24.04 LTS AMI
data "aws_ssm_parameter" "ubuntu_ami" {
name = "/aws/service/canonical/ubuntu/server/24.04/stable/current/amd64/hvm/ebs-gp3/ami-id"
}
9. Create Ubuntu EC2 Instance
resource "aws_instance" "web" {
ami = data.aws_ssm_parameter.ubuntu_ami.value
instance_type = "t3.micro"
key_name = "my_server"
Attach the existing Network Interface
primary_network_interface {
network_interface_id = aws_network_interface.web.id
}
Install and start Apache2 automatically
user_data = <<-EOF
Problems
Debug Console
Terminal
Ports
Output
Terminal
aws_instance.web: Still creating... [00m20s elapsed]
aws_instance.web: Creation complete after 22s [id=i-0f81be764f2958b9a]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Outputs:
apache_url = "http://35.168.220.185"
ec2_instance_id = "i-0f81be764f2958b9a"
elastic_ip = "35.168.220.185"
elastic_ip_allocation_id = "eipalloc-08c275d00997fb451"
internet_gateway_id = "igw-0904f04672653137"
network_interface_id = "eni-0d43646d165f8b76e"
network_interface_private_ip = "10.100.1.10"
Chat
Sessions
Describe what to build
Local
master*
Ln 15, Col 2
Spaces: 4
UTF-8
CRLF
Terraform
Sign In


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances:v=3;instanceState=running
Finish update
Apps
YouTube
Google
12 Beginner Python...
Learn Python
Learn Data Structur...
LeetCode - The Wor...
Movierulz | Watch B...
Slack
Search
[Alt+S]
United States (N. Virginia)
chandrashekar (986881030460)
chandrashekar
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Instances (1/1) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Running
Filter Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4
terraform-final-ubuntu-web
i-0f81be764f2958b9a
Running
t3.micro
3/3 checks passec
us-east-1a
ec2-35-168
i-0f81be764f2958b9a (terraform-final-ubuntu-web)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Instance summary Info
Instance ID
i-0f81be764f2958b9a
IPv6 address
-
Hostname type
Public IPv4 address
35.168.220.185 | open address
Instance state
Running
Private IP DNS name (IPv4 only)
Private IPv4 addresses
10.100.1.10
Public DNS
ec2-35-168-220-185.compute-1.amazonaws.com | open address
Answer private resource DNS name
CloudShell
Feedback
Console Mobile App
Privacy
Terms
Cookie preferences


us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#vpcs:
Search
[Alt+S]
United States (N. Virginia)
chandrashekar (986881030460)
VPC > Your VPCs
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
Carrier gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
AWS now provides personalized help through Amazon Q
Click Q help to receive personalized guidance for console tasks through Q Chat. To deactivate, click Deactivate Q help to access your console settings. Once deactivated, you'll receive static help content when available.
Launch Q help
Your VPCs
VPCs
VPC encryption controls
Your VPCs (1/3)
Q help
Last updated 2 minutes ago
Actions
Create VPC
Filter VPCs by attribute or tag
Name
VPC ID
State
Encryption c...
Encryption control ...
Block Public...
ASG-VPC
vpc-0978a6639ef05cc56
Available
Off
vpc-09ace864d17ee9449
Available
Off
terraform-final-vpc
vpc-0a37fd33813d62fcc
Available
Off
vpc-0a37fd33813d62fcc / terraform-final-vpc
Details
Resource map
CIDRs
Flow logs
Tags
Related resources
Integrations
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search
[Alt+S]
United States (N. Virginia)
chandrashekar (986881030460)
VPC > Route tables
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
Carrier gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Route tables (7)
Q help
Last updated 3 minutes ago
Actions
Create route table
Find route tables by attribute or tag
Name
Route table ID
Explicit subnet associ...
Edge associations
Main
VPC
rtb-0eabdcd7997f86214
Yes
vpc-0a37fd33813d62fcc | terraf
rtb-01de49626715e7f7b
Yes
vpc-09ace864d17ee9449
rtb-012f1dc369f9e9c33
Yes
vpc-0978a6639ef05cc56 | ASG-V
ASG-Private-RT
rtb-03db4c23b1d281826
2 subnets
No
RDS-Pvt-rt
rtb-0caac4d5e345c35f0
5 subnets
No
terraform-final-public-rt
rtb-01e7e338290b704ca
subnet-04c5ab259d6b0a...
No
ASG-Public-RT
rtb-09ef072657e7050b7
3 subnets
No
Select a route table
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



Terraform Final Task - Apache is running!